

Agenda

- Classification Models - Ensemble Methods (XGBoost)
- Naïve Bayes
- Imbalanced Classification
- Under sampling
- over sampling

3. Classification Models - XGBoost

XGBoost Classification Algorithm:

- The **XGBoost** “**Extreme Gradient Boosting**” algorithm is a decision tree based machine learning algorithm, which uses a process called boosting to help improve performance.
- It's one of the most effective machine learning algorithms and regularly produces results that outperform most other algorithms.
- It can be used to solve classification and regression problems.

What makes XGBoost so popular?

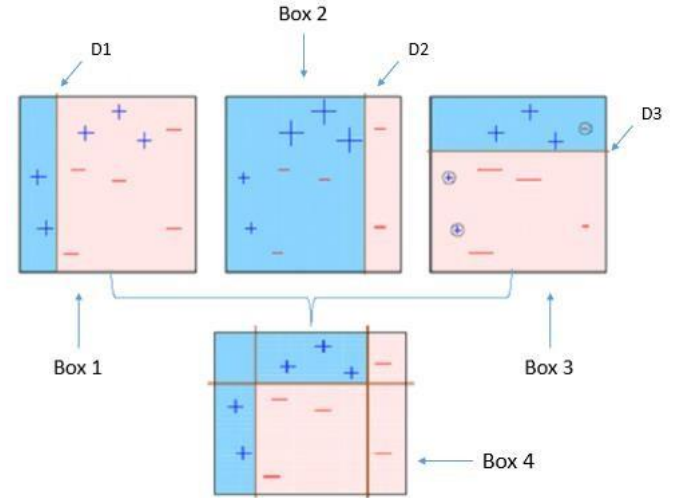
- **Speed and performance:** Originally written in C++, it is comparatively faster than other ensemble classifiers.
- **Core Algorithm is Parallelizable:** It can harness the power of multi-core computers. It is also parallelizable onto GPU's making it feasible to train on very large datasets as well.
- **Consistently Outperforms other Algorithm Methods:** It has shown better performance on a variety of machine learning benchmark datasets.
- **Wide Variety of Tuning Parameters:** XGBoost internally has parameters for cross-validation, regularization, user-defined objective functions, missing values, tree parameters, scikit-learn compatible API etc.

Boosting:

- Boosting is a sequential technique which works on the principle of an ensemble.
- It combines a set of weak learners and delivers improved prediction accuracy.
- At any instant t , the model outcomes are weighed based on the outcomes of previous instant $t-1$.
- The outcomes predicted correctly are given a lower weight and the ones miss-classified are weighted higher.

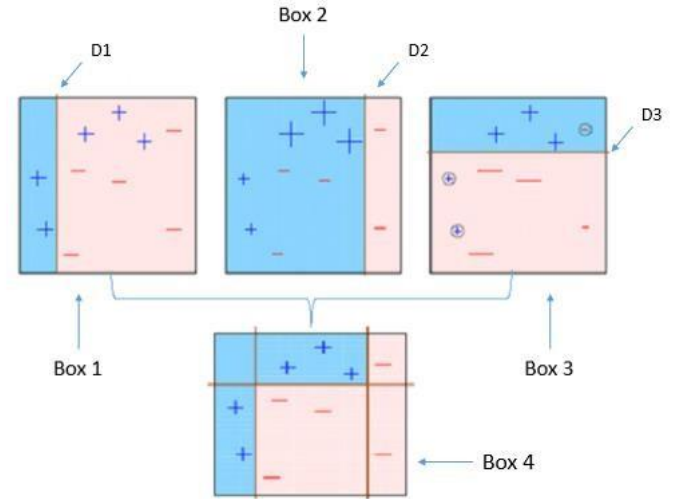
Boosting Example:

- **Box 1:** The first classifier creates a vertical line (split) at D1. It says anything to the left of D1 is + and anything to the right of D1 is -. However, this classifier misclassifies three + points.
- Note this is a Decision Tree model that only splits off at one level, therefore the final prediction is based on only one feature.



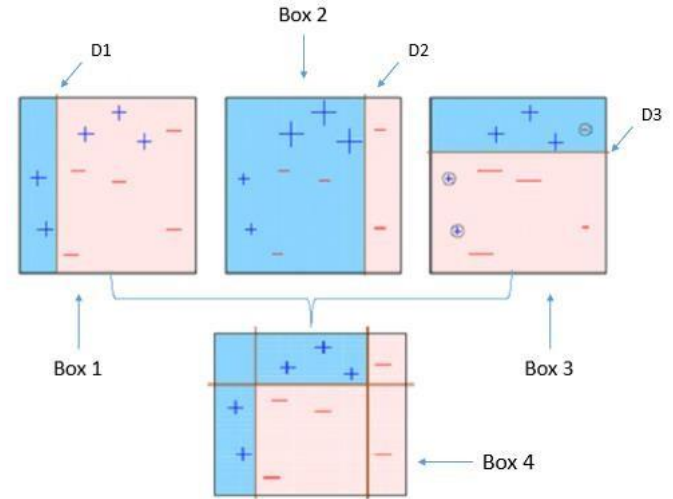
Boosting Example:

- **Box 2:** The second classifier gives more weight to the three + misclassified points (see the bigger size of +) and creates a vertical line at D2.
 - Again it says, anything to the right of D2 is - and left is +.
- Still, it makes mistakes by incorrectly classifying three - points.



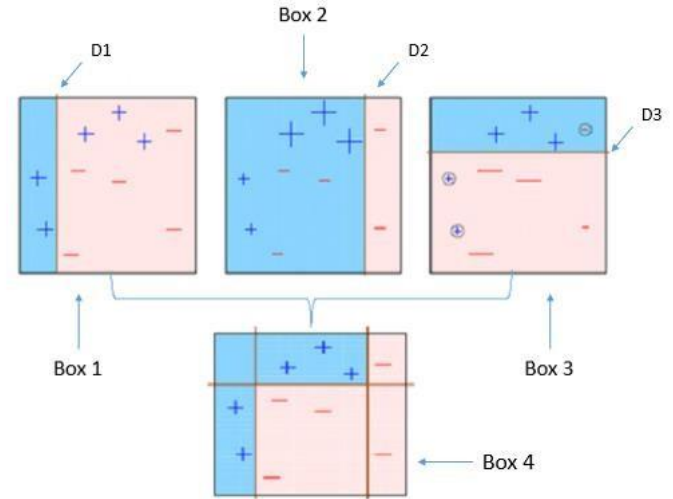
Boosting Example:

- **Box 3:** Again, the third classifier gives more weight to the three - misclassified points and creates a horizontal line at D3.
- Still, this classifier fails to classify the points (in the circles) correctly.



Boosting Example:

- **Box 4:** This is a weighted combination of the weak classifiers (Box 1, 2 and 3).
- As you can see, it does a good job at classifying all the points correctly.



Boosting Conclusion:

- The basic idea behind boosting algorithms is building a weak model.
- Making conclusions about the various feature importance and parameters.
- Then, using those conclusions to build a new, stronger model.
- Capitalize on the misclassification error of the previous model and try to reduce it.

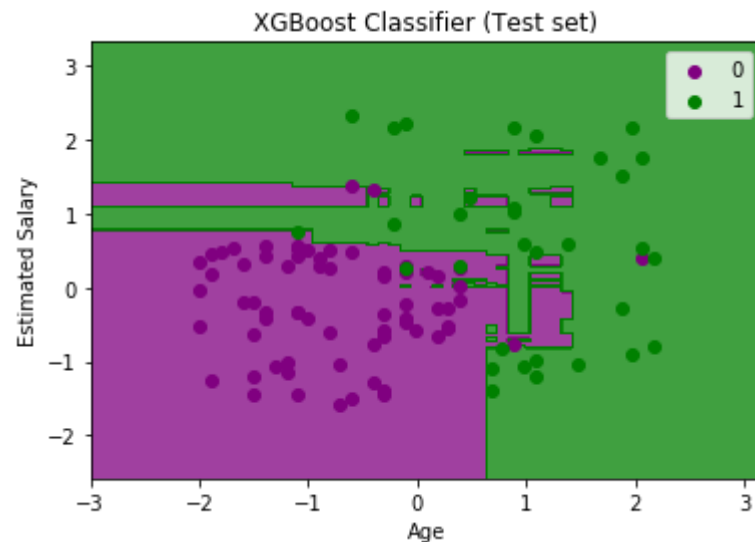
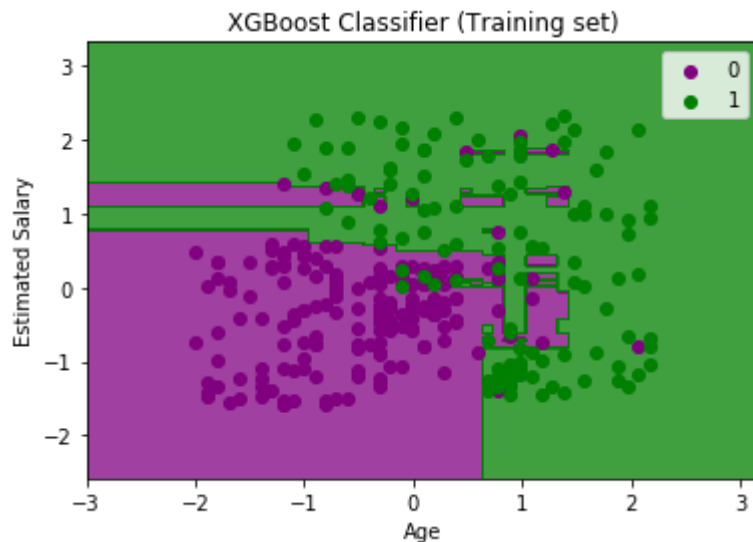
XGBoost:

- The default base learners of XGBoost is **Tree Ensembles**.
- The Tree Ensemble model is a set of classification and regression trees (CART).
- Trees are grown one after another, and attempt to reduce the misclassification rate made in subsequent iterations.
- Tree gives a different prediction score depending on the data it sees.
- The scores of each individual tree are summed up to get the final score.

XGBoost Important Parameters:

- eta: learning rate - step size shrinkage used to prevent overfitting.
- max_depth: determines how deeply each tree is allowed to grow during any boosting round.
- subsample: ratio of training data subsample used per tree.
- colsample_bytree: ratio of features used per tree.
- n_estimators: number of trees you want to build.
- alpha: L1 regularization on leaf weights.
- lambda: L2 regularization on leaf weights.
- gamma: controls whether a given node will split, based on the expected reduction in loss after the split.

XGBoost Output:



5. Naïve Bayes

Bayes theorem :

Bayes' Theorem is central to Bayesian Statistics. It allows for: (1) the updating prior probability distribution with a likelihood function based on new information, and (2) the calculation of the conditional probability, $P(A|B)$, given another calculated conditional probability, $P(B|A)$. Did you know that it is derived from basic probability logic?

Rule of Multiplication:

$$P(B \cap A) = P(A|B) P(B)$$

$$P(A \cap B) = P(B|A) P(A)$$

It follows that:

$$P(B \cap A) = P(A \cap B)$$

Therefore when we substitute we get:

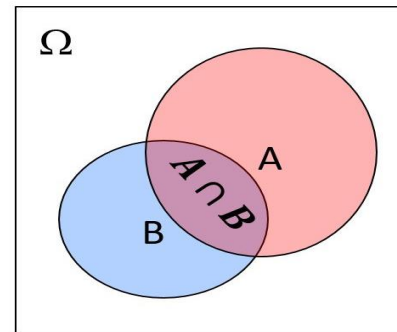
$$P(A|B) P(B) = P(B|A) P(A)$$

Now we have Bayes' Theorem!

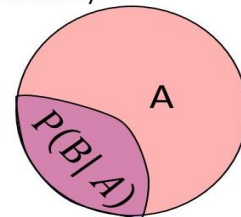
$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

The terms are known as:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$



Venn Diagram - Marginal and joint probability



Venn Diagram - Marginal and conditional probability

Naïve Bayes:

- **Naïve Bayes:** It is a simple and powerful algorithm for predictive modelling.
- It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.
- It is based on **Bayes' theorem** and used for solving classification problems.
- Naïve Bayes is called naive because it assumes that each input variable is independent.
- This is a strong assumption and unrealistic for real data; however, the technique is very effective on a large range of complex problems.

Why is it called Naïve Bayes?

- **Naïve:** It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of colour, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes:** It is called Bayes because it depends on the principle of Bayes' Theorem.

Bayes' Theorem:

- It is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where,

- $P(A|B)$ is Posterior Probability: Probability of hypothesis A on the observed event B.
- $P(B|A)$ is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.
- $P(A)$ is Prior Probability: Probability of hypothesis before observing the evidence.
- $P(B)$ is Marginal Probability: Probability of Evidence.

Working of Naïve Bayes' Classifier:

- Suppose we have a dataset of weather conditions and corresponding target variable "Play".
- So using this dataset we need to decide either we should play or not on a particular day according to the weather conditions.
- So to solve this problem, we need to follow the below steps:
 1. Convert the given dataset into frequency tables.
 2. Generate Likelihood table by finding the probabilities of given features.
 3. Now, use Bayes' Theorem to calculate the posterior probability.

Working of Naïve Bayes' Classifier:

- Problem: If the weather is sunny, then the Player should play or not?
- Solution: To solve this,
first consider this dataset:

$$\underline{P(\text{Play} \mid \text{Outlook}) = ?}$$

$$P(\text{Yes} \mid \text{Outlook}) = ?$$

$$P(\text{No} \mid \text{Outlook}) = ?$$

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Working of Naïve Bayes' Classifier:

→ Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	4

→ Likelihood table for the Weather Conditions:

Weather	Yes	No	
Overcast	5	0	$5/14 = 0.35$
Rainy	2	2	$4/14 = 0.29$
Sunny	3	2	$5/14 = 0.35$
All	$10/14 = 0.71$	$4/14 = 0.29$	

Working of Naïve Bayes' Classifier:

Applying Bayes' Theorem:

- $P(\text{Yes} | \text{Sunny}) = P(\text{Sunny} | \text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$
- $P(\text{Sunny} | \text{Yes}) = 3/10 = 0.3$
- $P(\text{Sunny}) = 0.35$
- $P(\text{Yes}) = 0.71$

$$\text{So } P(\text{Yes} | \text{Sunny}) = 0.3 * 0.71 / 0.35 = 0.60$$

- $P(\text{No} | \text{Sunny}) = P(\text{Sunny} | \text{No}) * P(\text{No}) / P(\text{Sunny})$
- $P(\text{Sunny} | \text{No}) = 2/4 = 0.5$
- $P(\text{Sunny}) = 0.35$
- $P(\text{No}) = 0.29$

$$\text{So } P(\text{No} | \text{Sunny}) = 0.5 * 0.29 / 0.35 = 0.41$$

So, $P(\text{Yes} | \text{Sunny}) > P(\text{No} | \text{Sunny}) \rightarrow$ on a Sunny day, Player can play the game.

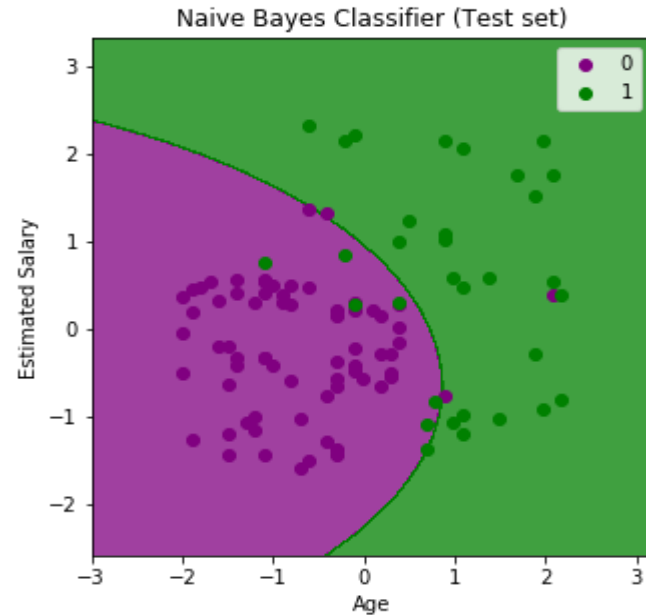
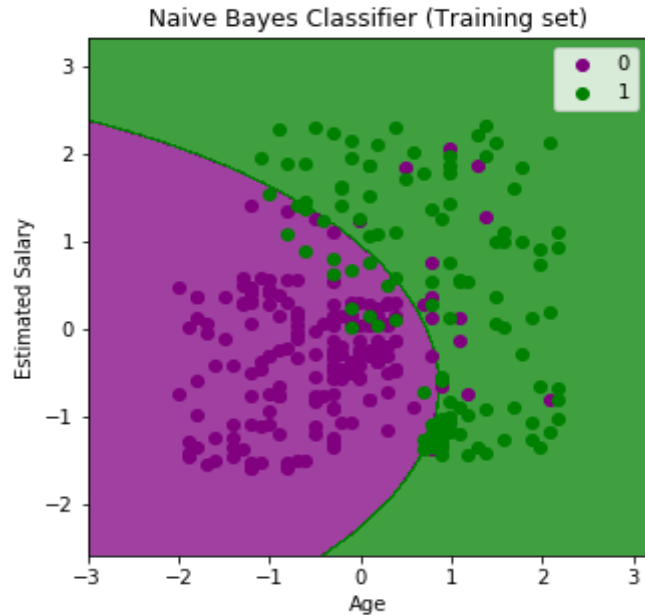
Advantages of Naïve Bayes' Classifier:

- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
- It can be used for Binary as well as Multi-class Classifications.
- It performs well in Multi-class predictions as compared to the other Algorithms.
- It is the most popular choice for text classification problems.

Disadvantages of Naïve Bayes' Classifier:

- Naïve Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

Sklearn Naïve Bayes:

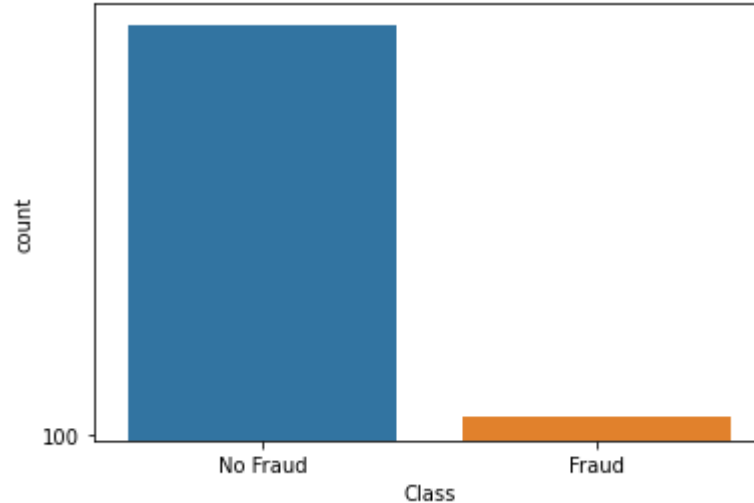


→ As we can see the classifier has created a Gaussian curve to divide the "purchased" and "not purchased" variables.

6. Imbalanced Classification

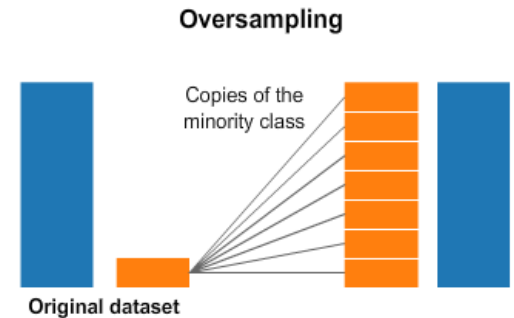
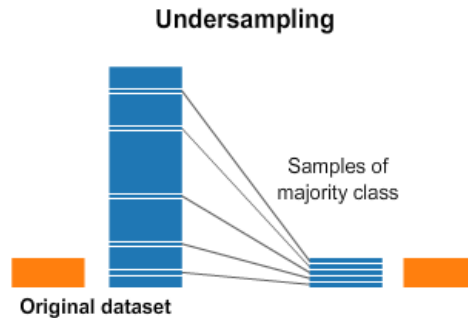
Imbalanced Classification:

- Imbalanced datasets are those where there is a severe skew in the classes distribution.
- This bias in the training dataset can influence many machine learning algorithms, leading some to ignore the minority class entirely.
- This is a problem as it is typically the minority class on which predictions are most important.



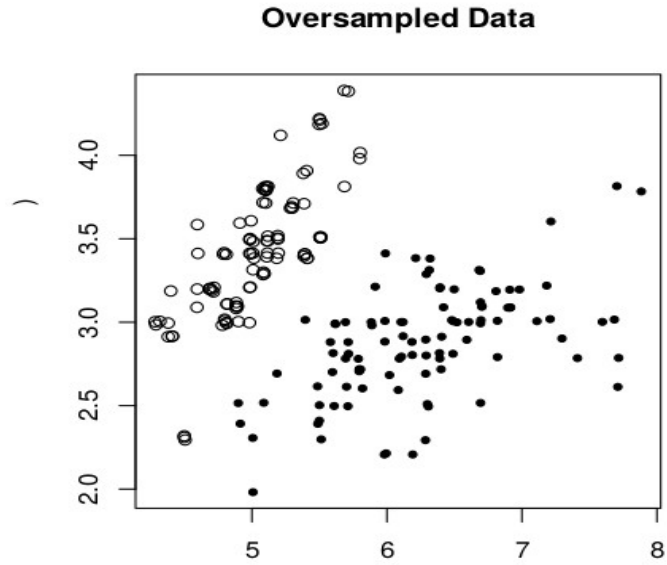
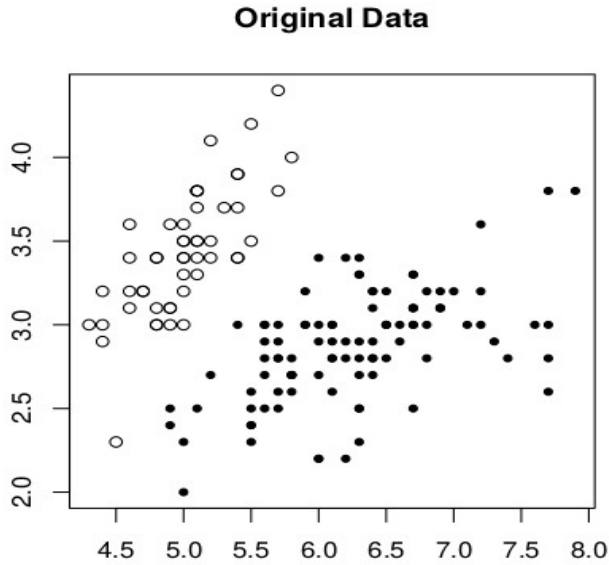
Imbalanced Data Pre-Processing :

- Data Pre-Processing strategies are also known as Resampling Strategies - These are the most common strategies to tackle imbalanced domain learning task
- Over sampling:
 - Random Over sampling
 - SMOTE
- Under sampling:
 - Random Under sampling
 - Tomek links



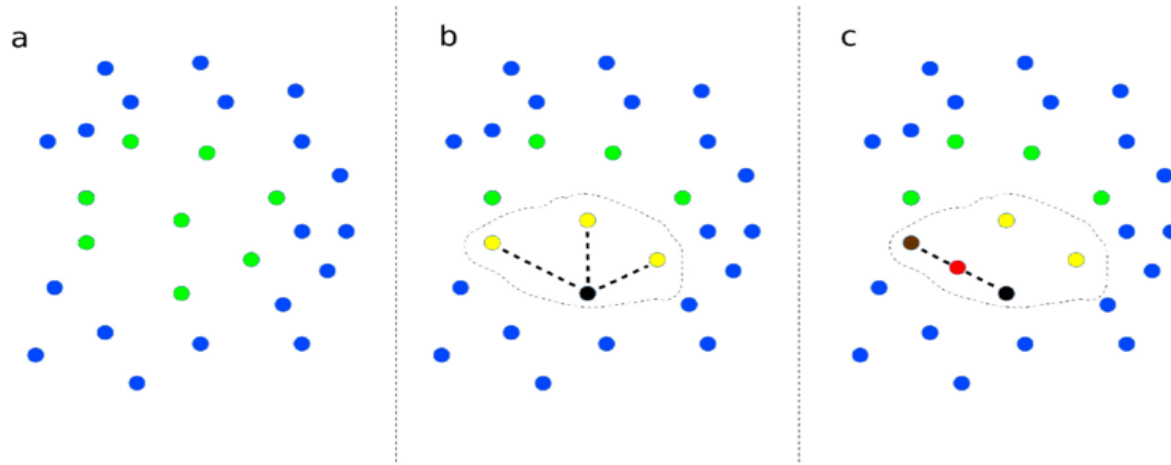
Random Over-sampling

→ **Random oversampling** duplicates examples from the minority class in the training dataset and can result in overfitting for some models.

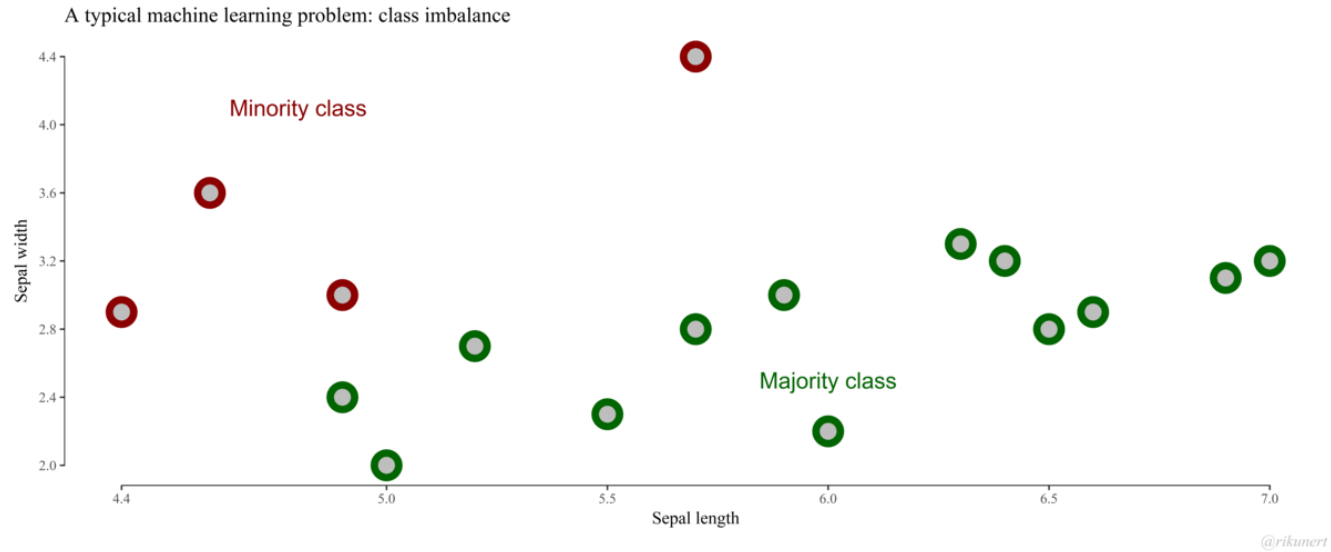


SMOTE

Synthetic Minority Oversampling Technique (SMOTE) works by selecting examples that are close in the feature space, drawing a line between the examples in the feature space and drawing a new sample at a point along that line.

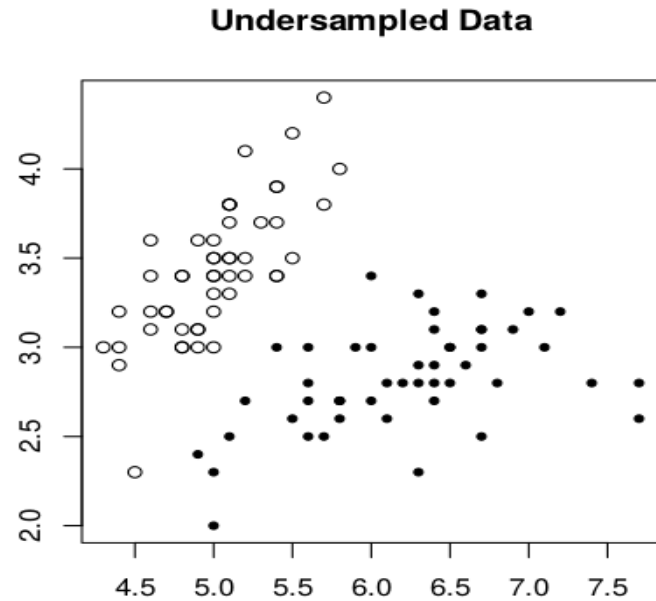
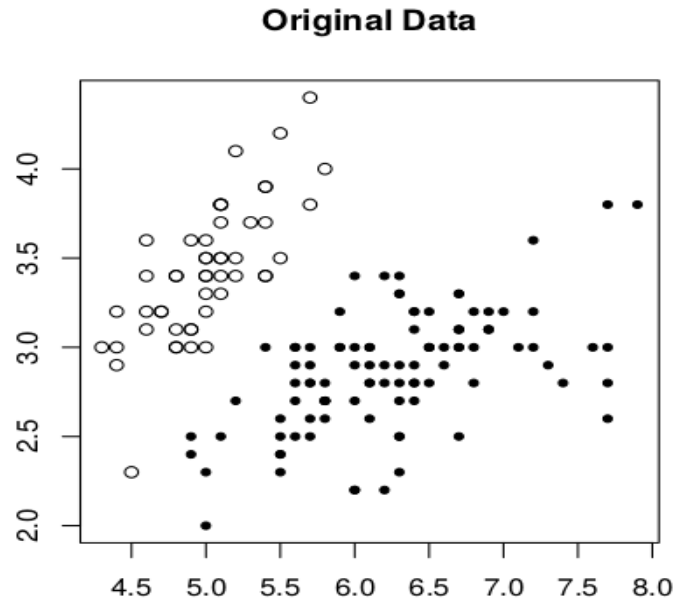


SMOTE



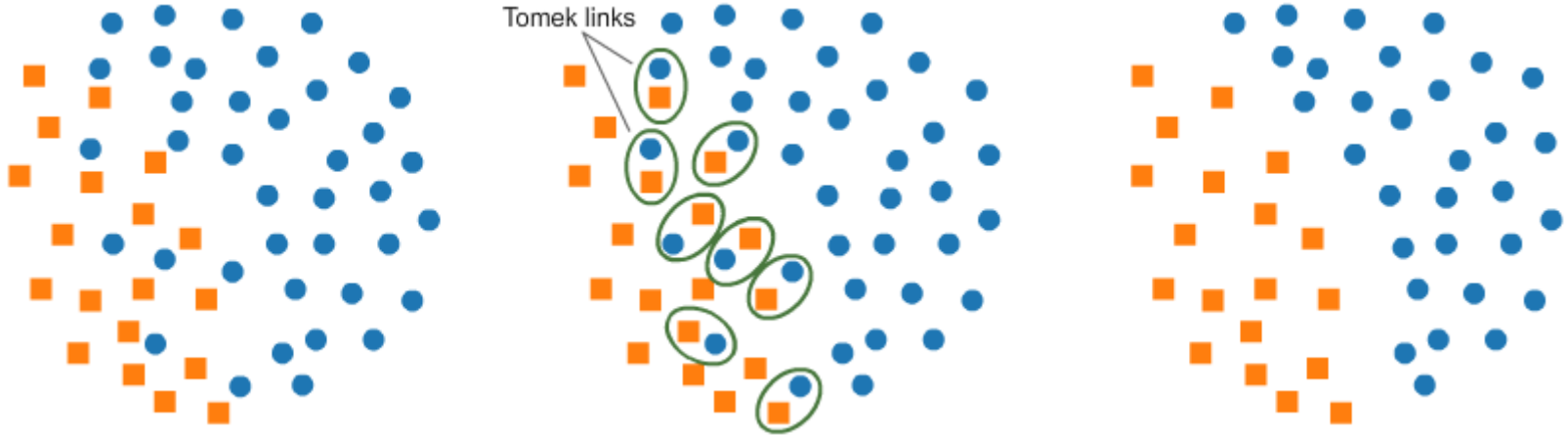
Random under - sampling

Random under – sampling Random under sampling deletes examples from the majority class and can result in losing information invaluable to a model.



Tomek links

Tomek's algorithm looks for such pairs and removes the majority instance of the pair. The idea is to clarify the border between the minority and majority classes, making the minority region(s) more distinct.



Imbalanced Classification:

Over Sampling:

- Random oversampling duplicates examples from the minority class in the training dataset and can result in overfitting for some models.

Under Sampling:

- Random undersampling deletes examples from the majority class and can result in losing information invaluable to a model.